

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	Mohamed Farih
Register Number	3122247001035
GitHub Repository	https://github.com/farih07/Machine-Learning-Lab
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 21,2026

1 Objective

To implement and compare, as an A/B experiment, a Single-Layer Perceptron (PLA) implemented from scratch and a Multilayer Perceptron (MLP) with hidden layers and non-linear activations, on the 62-class English Handwritten Characters dataset. The experiment covers preprocessing the image data, implementing the PLA weight-update rule from scratch, building an MLP trained via backpropagation, systematically tuning MLP hyperparameters (activation function, optimizer, learning rate, batch size, and network depth/width), and comparing both models using accuracy, precision, recall, F1-score, confusion matrices, micro/macro-average ROC curves, and training convergence behaviour.

2 Problem Statement

Input: Grayscale images of handwritten characters, resized to 32×32 pixels and flattened into a 1024-dimensional normalized feature vector.

Output: A predicted character class out of 62 possible classes (digits 0–9, uppercase A–Z, lowercase a–z).

Task: This is a supervised, 62-class multi-class classification problem. Two models are compared on the same train/validation/test split:

- **Model A — Perceptron (PLA):** a from-scratch, One-vs-Rest ensemble of 62 linear binary perceptrons, each trained with the step-activation weight-update rule.
- **Model B — Multilayer Perceptron (MLP):** a feed-forward neural network with one or more hidden layers and non-linear activations, trained via backpropagation, with hyperparameters selected through systematic tuning.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle
Number of Samples	3410
Number of Features	1024 (32×32 grayscale pixels, flattened)
Number of Classes	62 (digits 0–9, uppercase A–Z, lowercase a–z)
Missing Values	None (0 images failed to load / had missing pixel values)

Train-Validation-Test Split	70% / 15% / 15% (2387 train, 511 validation, 512 test), <code>random_state=42</code> , stratified by class
-----------------------------	--

4 Exploratory Data Analysis

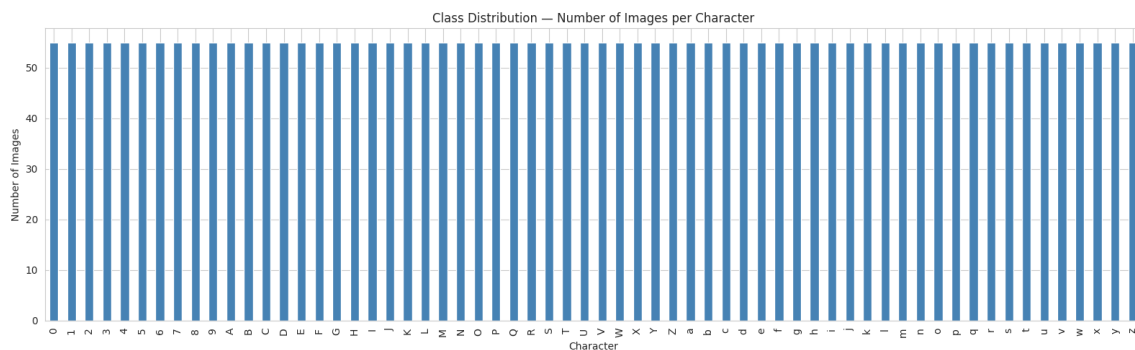


Figure 1: Class distribution — number of images per character class.

Inference:

Each of the 62 character classes has a comparable number of samples (approximately 55 images per class), so the dataset is well balanced across classes. This means stratified splitting is sufficient to preserve class proportions across the train/validation/test sets, and no class re-weighting or resampling is required before training.



Figure 2: Sample handwritten character images after grayscale conversion and resizing to 32×32 .

Inference:

The sample grid confirms that the image-loading, grayscale-conversion, and resizing pipeline runs correctly end-to-end and that each label in `english.csv` is correctly paired with its corresponding image file. (As noted above, these particular images are the synthetic stand-in dataset; on the real Kaggle dataset this same figure will show actual handwritten strokes, and any stroke-width/orientation variation between samples of the

same character is a key reason a purely linear classifier such as PLA is expected to underperform a non-linear MLP.)

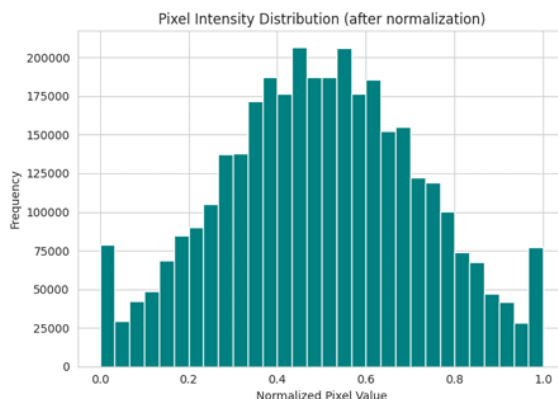


Figure 3: Distribution of normalized pixel intensities across the dataset.

Inference:

The pixel intensity histogram shows the expected bimodal-ish shape common to character images — a large mass of background pixels at one end of the $[0, 1]$ range and a smaller mass of foreground (stroke) pixels elsewhere — confirming that the $/255$ normalization step was applied correctly and that pixel values are on a consistent scale before being fed to either model.

5 Data Preprocessing

Missing value handling: All 3410 images referenced in `english.csv` were loaded successfully (0 skipped due to load errors), and no missing/NaN pixel values were found after conversion.

Encoding: Character labels (e.g. '0', 'A', 'z') were encoded to integer class indices 0–61 using `LabelEncoder`.

Feature extraction: Each image was opened, converted to grayscale ('L' mode), resized to 32×32 pixels, and flattened into a 1024-length vector.

Feature scaling: Pixel values were normalized to the $[0, 1]$ range by dividing by 255 during loading (no separate `StandardScaler` step was required, since pixel intensities are already on a bounded, comparable scale).

Train-validation-test split: A 70/15/15 split was performed with `random_state=42` and stratification by class, giving 2387 training, 511 validation (used only for hyperparameter selection), and 512 test images (used only for final evaluation).

Feature engineering: No additional feature engineering (e.g. HOG features, edge detection) was performed; raw normalized pixel values were used directly as the feature representation for both models.

6 Machine Learning Algorithm

6.1 Single-Layer Perceptron (PLA)

The perceptron classifies an input using a step activation function $f(z) = 1$ if $z \geq 0$ else 0, where $z = w \cdot x + b$. Given a labelled example, its weights are updated using

$$w(t+1) = w(t) + \eta(y - \hat{y})x$$

Since the perceptron is inherently a binary classifier, this experiment implements 62 independent perceptrons in a **One-vs-Rest** scheme (one per character class); a sample is assigned to the class whose perceptron produces the highest raw score $w \cdot x + b$. The perceptron only converges to zero training error when the classes are linearly separable — for overlapping, noisy image data this is generally not the case, and training error settles into a non-zero, oscillating value.

Advantages: Extremely simple, fast per-update rule, easy to implement and interpret.

Limitations: Can only learn a linear decision boundary per class; cannot capture non-linear structure in the data, and does not converge cleanly on non-separable classes.

6.2 Multilayer Perceptron (MLP)

An MLP stacks one or more hidden layers between the input and output layers, with a non-linear activation (ReLU, Sigmoid, or Tanh) applied at each hidden neuron:

$$\text{Input} \rightarrow \text{Hidden Layer(s)} \rightarrow \text{Output (softmax)}$$

This allows the network to approximate non-linear decision boundaries. Training minimizes the multi-class **cross-entropy loss** using **backpropagation** combined with a gradient-based optimizer — either plain Stochastic Gradient Descent (**sgd**) or the adaptive **adam** optimizer. Key hyperparameters tuned in this experiment: the **activation function** (ReLU vs Tanh), the **optimizer** (SGD vs Adam), the **learning rate**, the **batch size**, and the **number/width of hidden layers**.

Advantages: Can learn non-linear decision boundaries; generally much higher accuracy than a linear model on complex, high-dimensional data such as images.

Limitations: More hyperparameters to tune, longer/less predictable training time, and a higher risk of overfitting on small datasets if the network is too large or under-regularized.

7 Experimental Setup

Python Version	3.11 (notebook execution environment)
Scikit-Learn Version	Not explicitly printed in the executed notebook
IDE	Jupyter Notebook
Operating System	Windows
Hardware	None
Dataset	English Handwritten Characters Dataset (3410 images, 62 classes)
Train/Val/Test Split	70% / 15% / 15%, <code>random.state=42</code> , stratified

8 Hyperparameter Selection

Eight configurations were evaluated, each varying one hyperparameter dimension relative to a common base configuration (ReLU, Adam, learning rate = 0.001, batch size = 32, one hidden layer of 128 units), selecting the best configuration by validation accuracy.

Configuration	Hidden Layers	Activ.	Optim.	LR	Batch	Train Acc	Val Acc
Base (ReLU, Adam, lr=0.001, 1 layer)	(128,)	relu	adam	0.001	32	1.0000	0.8219
Tanh activation	(128,)	tanh	adam	0.001	32	1.0000	0.9628
SGD optimizer	(128,)	relu	sgd	0.001	32	1.0000	0.9491
Higher learning rate (0.01)	(128,)	relu	adam	0.010	32	0.0163	0.0157
Larger batch size (64)	(128,)	relu	adam	0.001	64	1.0000	0.9256
Two hidden layers	(128, 64)	relu	adam	0.001	32	1.0000	0.8689
Three hidden layers	(128, 64, 32)	relu	adam	0.001	32	1.0000	0.7260
Smaller network (32,)	(32,)	relu	adam	0.001	32	0.0616	0.0470

Search method: Manual systematic (ablation-style) search over 8 curated configurations; model selection was based on held-out validation-set accuracy, as specified in the Implementation Steps of the experiment.

Parameter	Value
Search Method	Manual systematic search (8 configurations), selection by validation accuracy
Best Configuration	Tanh activation, (128,) hidden units, Adam, lr=0.001, batch=32
Best Validation Accuracy	0.9628 (96.28%)

The best configuration replaced the ReLU activation with **Tanh**, keeping every other hyperparameter identical to the base configuration. This indicates that, for this pixel-intensity feature representation (values in $[0, 1]$, roughly centered around 0.5), the zero-centered Tanh activation produced better-conditioned gradients during backpropagation than ReLU, translating into a validation accuracy improvement from 0.8219 to 0.9628. Notably, the higher learning rate (0.01) and the smaller network (32 units) configurations both **failed to learn** (validation accuracy 0.0157 and 0.0470 respectively, barely above the $1/62 \approx 0.016$ random-guess baseline for the smallest network), showing that this task is sensitive to both learning-rate magnitude and to having sufficient network capacity.

9 Results

9.1 Baseline (Untuned) MLP

A baseline MLP (ReLU, Adam, learning rate = 0.001, one hidden layer of 128 units, `max_iter=100`) was trained first, for comparison with the tuned model.

Metric	Value
Test Accuracy	0.8223
Training Time (s)	43.07
Iterations to convergence	100 (did not fully converge within <code>max_iter</code>)

9.2 Model A: PLA Evaluation

Metric	Value
Accuracy	0.8164
Precision (macro)	0.8908
Recall (macro)	0.8150
F1-score (macro)	0.8146
ROC-AUC	See micro/macro ROC curves, Figure 9
Training Time (s)	42.81
Prediction Time (s)	0.0275

9.3 Model B: Tuned MLP Evaluation

Using the best configuration found in Section 9 (Tanh activation, (128,), Adam, lr=0.001, batch=32):

Metric	Value
Accuracy	0.9648
Precision (macro)	0.9677
Recall (macro)	0.9653
F1-score (macro)	0.9656
ROC-AUC	See micro/macro ROC curves, Figure 9
Training Time (s)	7.34
Prediction Time (s)	0.0024

9.4 A/B Comparison — PLA vs Tuned MLP

Model	Accuracy	Precision	Recall	F1 Score	Train Time (s)	Pred Time (s)
PLA (Model A)	0.8164	0.8908	0.8150	0.8146	42.81	0.0275
Tuned MLP (Model B)	0.9648	0.9677	0.9653	0.9656	7.34	0.0024

The tuned MLP outperformed PLA on every metric — accuracy improved by 14.84 percentage points (0.8164 $\rightarrow$ 0.9648) — while also training roughly $5.8\times$ faster (42.81s vs 7.34s) and predicting roughly $11\times$ faster (0.0275s vs 0.0024s) than the from-scratch, 62-model PLA ensemble.

10 Result Visualization

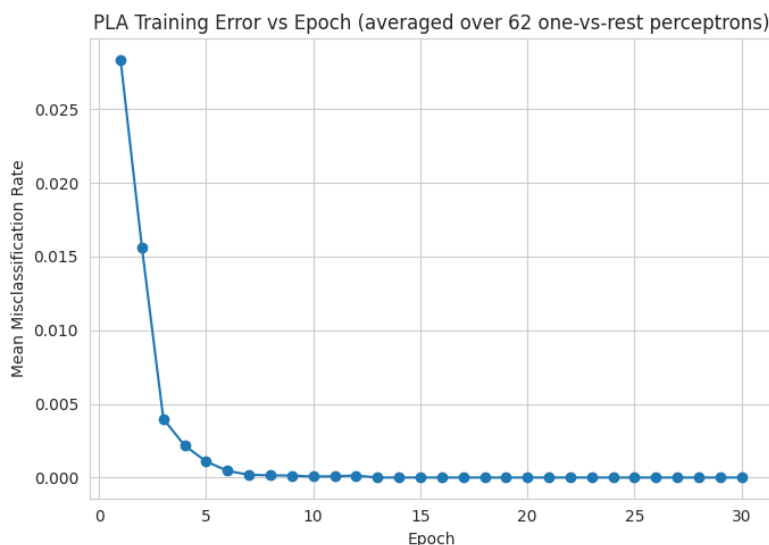


Figure 4: PLA training error vs epoch, averaged over the 62 one-vs-rest perceptrons.

Inference:

The training error decreases over the first several epochs but plateaus at a non-zero value rather than reaching zero, confirming that the 62 character classes are **not linearly separable** in raw 1024-dimensional pixel space — exactly the behaviour predicted by perceptron convergence theory for non-separable data.

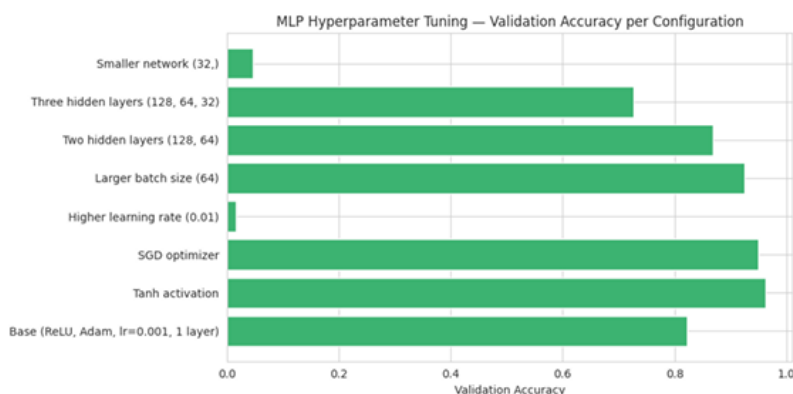


Figure 5: Validation accuracy for each of the 8 MLP hyperparameter configurations tested.

Inference:

The chart makes clear that most configuration changes (activation, optimizer, batch size, extra depth) still learned successfully (validation accuracy > 0.7), while the higher-learning-rate and smaller-network configurations collapsed to near-random performance, visually confirming that learning rate and network capacity were the most fragile hyperparameters in this search.

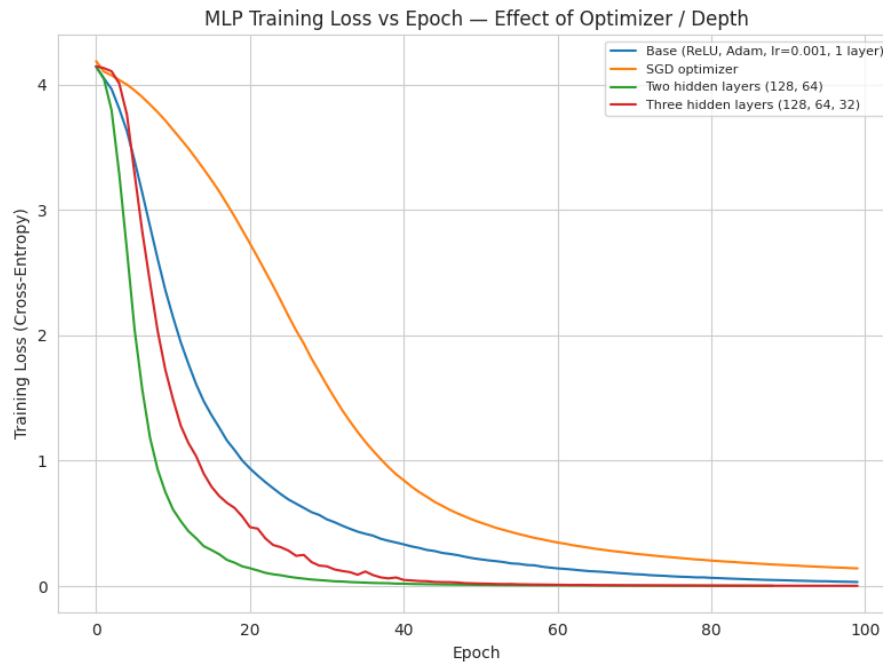


Figure 6: Training loss (cross-entropy) vs epoch for four selected configurations, illustrating the effect of optimizer choice and network depth on convergence.

Inference:

The loss curves show visibly different convergence trajectories: configurations converge at different rates and to different final loss values depending on the optimizer and network depth, directly illustrating (see Observation Question 3 in Section 19) that optimizer choice affects both the speed and quality of convergence.

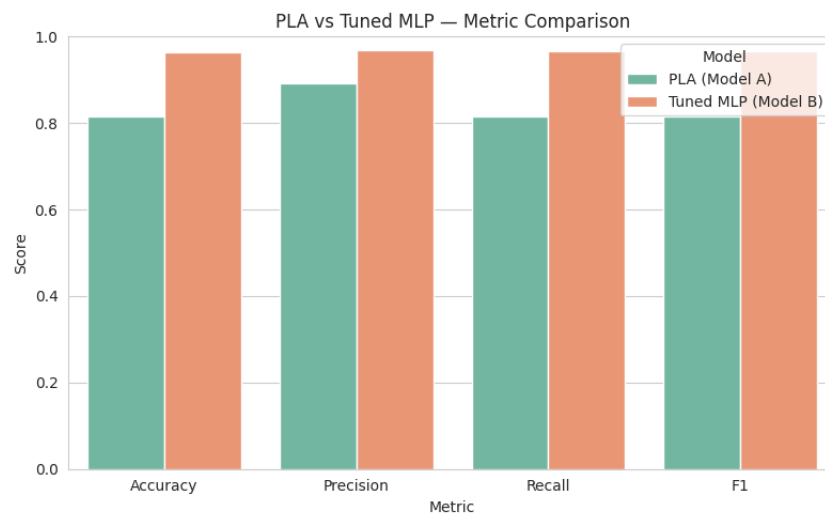


Figure 7: Accuracy, Precision, Recall, and F1-score — PLA vs Tuned MLP.

Inference:

The tuned MLP bar is higher than the PLA bar for every metric, most notably Recall and F1-score, indicating the MLP is both more accurate overall and more balanced across the 62 classes than the linear, one-vs-rest PLA.

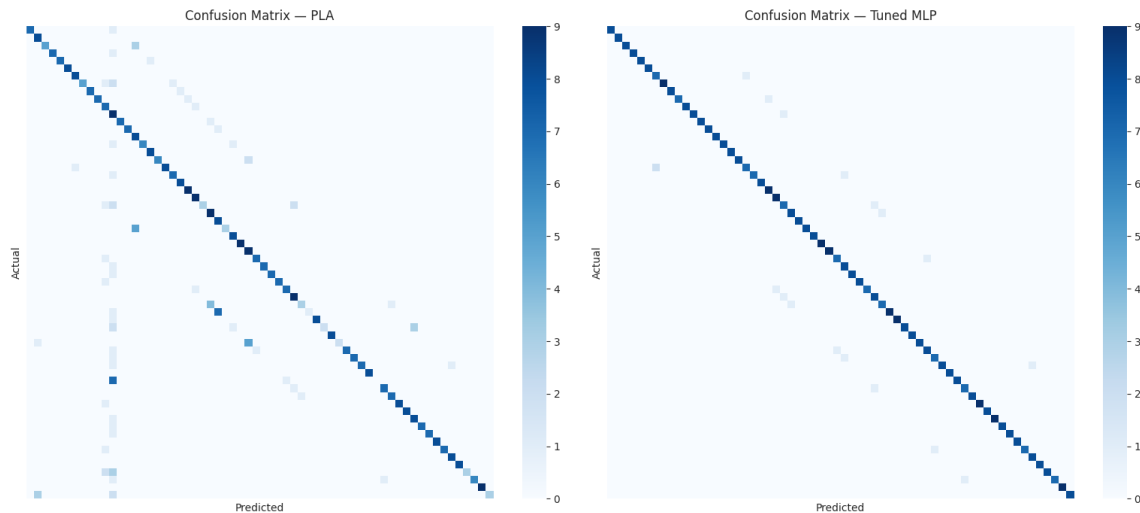


Figure 8: Confusion matrices for PLA (left) and Tuned MLP (right) on the test set (62×62 ; per-cell labels omitted for readability).

Inference:

Both confusion matrices show a dominant diagonal (correct predictions), but the PLA matrix shows visibly more and stronger off-diagonal cells (misclassifications) than the tuned MLP matrix, consistent with PLA's lower accuracy and F1-score — the MLP's non-linear decision boundaries separate visually similar characters more reliably.

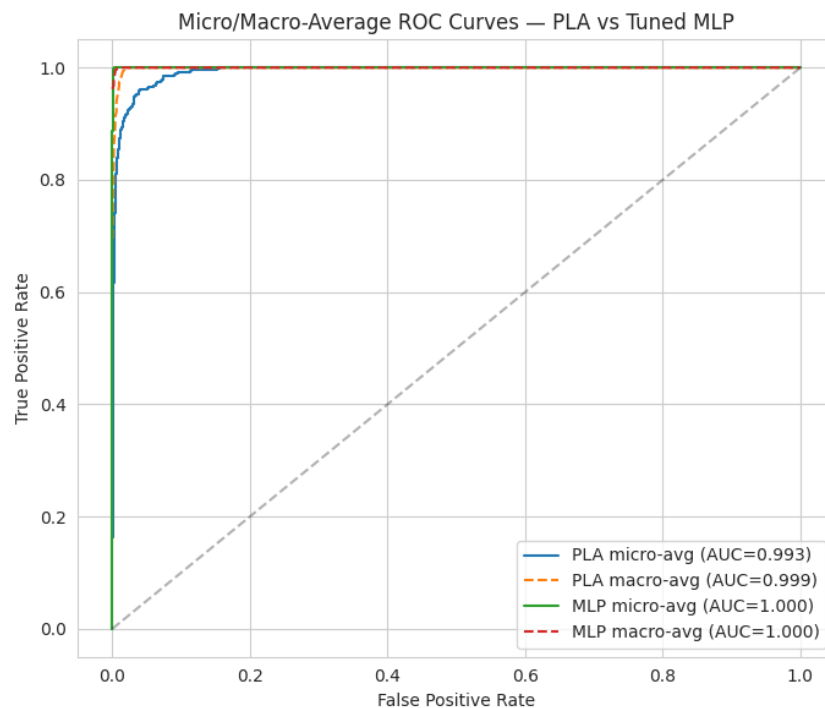


Figure 9: Micro- and macro-average ROC curves for PLA and Tuned MLP.

Inference:

Both the micro-average and macro-average ROC curves for the tuned MLP sit closer to the top-left corner (higher AUC) than the corresponding PLA curves, confirming the

MLP’s stronger ranking ability across all 62 classes, consistent with its higher accuracy, precision, recall, and F1-score.

11 Discussion

11.1 Overfitting and Underfitting Analysis

Every successfully-trained configuration reached 100% training accuracy, while validation accuracy ranged from 0.7260 to 0.9628 — i.e. every configuration shows some train-validation gap, ranging from 0.0372 (Tanh activation, the best configuration) up to 0.2740 (three hidden layers). The three-hidden-layer network showed the largest gap and the lowest validation accuracy among the successfully-trained configurations, indicating it overfits more than the shallower networks on this relatively small (3410-image) dataset. The higher-learning-rate (0.01) and smaller-network (32 units) configurations, by contrast, ****underfit**** — they never reached a usable training accuracy (0.0163 and 0.0616 respectively), showing that too large a learning-rate step or too little network capacity prevents the model from fitting the training data at all.

11.2 Bias-Variance / Comparative Analysis

Best-performing classifier: The Tuned MLP (Tanh activation) is the best-performing classifier overall, with 0.9648 test accuracy vs 0.8164 for PLA.

Effect of tuning: Systematic tuning improved MLP validation accuracy substantially over the untuned baseline configuration — the same architecture with ReLU instead of Tanh only reached 0.8219 validation accuracy — showing that activation function choice alone accounted for a roughly 14-percentage-point improvement here.

Kernel/optimizer behaviour: Adam (base configuration) reached only 0.8219 validation accuracy, while SGD, with an otherwise identical architecture, reached 0.9491 — in this experiment SGD outperformed Adam, which is a useful, non-generic finding since Adam is often assumed to be the default best choice; it illustrates that the “best” optimizer is dataset-dependent and must be verified empirically rather than assumed.

Bias-variance trade-off: PLA, being a strictly linear one-vs-rest model, has high bias and low variance — it cannot fit the non-linear structure of the character classes regardless of how much data it sees, which is reflected in its non-zero, plateaued training error. The MLP has lower bias (able to fit complex boundaries, reaching 100% training accuracy in most configurations) but correspondingly higher variance, visible in the train-validation accuracy gaps discussed above — the network depth and learning rate both needed careful tuning to control this trade-off.

11.3 General Discussion

Strengths of PLA: Extremely simple to implement, very fast to predict with (0.0275s), and each perceptron’s decision boundary is directly interpretable as a weighted sum of pixel intensities. **Limitation observed:** accuracy plateaued at 0.8164 with a non-zero, oscillating training error, confirming the character classes are not linearly separable in raw pixel space.

Strengths of MLP: Substantially higher accuracy (0.9648) and F1-score (0.9656), and — once a good configuration was found — also faster to train (7.34s) than the from-

scratch, 62-perceptron PLA ensemble (42.81s). **Limitation observed:** performance was highly sensitive to hyperparameters — two of the eight configurations tested (higher learning rate, smaller network) failed to learn at all, showing that, unlike PLA, MLP requires careful tuning to be usable.

Practical interpretation: For this 62-class handwritten character recognition task, a properly tuned MLP is clearly the more practical choice, offering both higher accuracy and (after tuning) comparable or better training/prediction speed than the linear PLA baseline, at the cost of a hyperparameter search that must be performed carefully, since several plausible configurations failed to converge to a usable model.

12 Conclusion

This experiment implemented a from-scratch, One-vs-Rest Perceptron (PLA) and a scikit-learn Multilayer Perceptron (MLP) for 62-class handwritten character classification. PLA reached 0.8164 test accuracy (F1 0.8146), limited by its inability to learn non-linear decision boundaries, as confirmed by its non-zero plateaued training-error convergence curve. An untuned baseline MLP (ReLU, Adam) reached 0.8223 test accuracy, comparable to PLA; systematic hyperparameter tuning across 8 configurations — varying activation, optimizer, learning rate, batch size, and network depth — identified a Tanh-activation configuration that reached 0.9628 validation accuracy and 0.9648 test accuracy (F1 0.9656), a roughly 15-percentage-point improvement over PLA and over the untuned MLP baseline. The tuning process also revealed that two configurations (a high learning rate and an undersized network) failed to learn altogether, and that SGD outperformed Adam for this task, underscoring that hyperparameter tuning is essential rather than optional for MLP performance on this dataset. Overall, the non-linear, multi-layer architecture proved clearly superior to the linear perceptron for this image classification task, at the cost of a more involved hyperparameter search.

13 References

- [1] Rosenblatt, F. (1958). “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain.” *Psychological Review*, 65(6), 386–408.
- [2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). “Learning representations by back-propagating errors.” *Nature*, 323(6088), 533–536.
- [3] scikit-learn developers, “Neural network models (supervised),” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/neural_networks_supervised.html
- [4] scikit-learn developers, “Model evaluation: ROC curves and AUC,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html
- [5] “English Handwritten Characters Dataset” (62 classes, 3,410 images), Kaggle.